SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Industry Problem Solving Task (Medium)

Assessment Tool Weightage: 35%

Dr

QUESTIONS Marks: 50			Total
Q.No	**Question**	**Bloom's Level**	**Marks**
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	**5**
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	**5**
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	**5**

4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	**5**
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	**5**
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	**5**
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	**5**
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	**5**
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	**5**
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	**5**

Code of Conduct:

I D. Gnana Deepthi (192411123)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANSWERS

1. File Organization Strategy for an E-Commerce Product Database

Introduction

An e-commerce company needs to store 10 million product records. The file organization method should provide fast product searching, efficient insertion and deletion, and reasonable storage and maintenance costs. For this requirement, B+ Tree-based indexed file organization is a suitable choice.

Recommended Strategy: Indexed File Organization

The product records can be stored in a **heap or sequential data file**, while a **B+ Tree index** is maintained on frequently searched attributes such as Product_ID, Category, or Product_Name.

For example:

Product File:

Product_ID | Product_Name | Category | Price | Stock

B+ Tree Index:

Product_ID → Record/Block Address

The B+ Tree keeps the index sorted and allows the system to quickly locate the required product without scanning all 10 million records.

Why B+ Tree is Suitable

- **Fast searching:** Search takes approximately **O(log N)**.
- **Efficient range queries:** Products within a price range or product-ID range can be retrieved efficiently.
- **Scalable:** Suitable for millions of records.
- **Dynamic:** Insertions and deletions can be performed without reorganizing the entire file.
- **Good disk utilization:** B+ Trees have high fan-out, reducing the number of disk I/O operations.
- **Supports e-commerce operations:** Useful for product lookup, category filtering, price-range searches, and inventory queries.

Cost Analysis

Assume:

- Number of records = **10,000,000**

- Average record size = **200 bytes**
- Block size = **4 KB = 4096 bytes**

Approximate data storage:

$$
10,000,000 \times 200 = 2,000,000,000\text{ bytes}
$$

Therefore, the product file requires approximately:

$$
\frac{2,000,000,000}{4096} \approx 488,282\text{ blocks}
$$

Sequential File Organization

If records are stored sequentially, searching for a product that is not indexed may require scanning many blocks.

Worst-case I/O:

$$
\approx 488,282\text{ block accesses}
$$

This is expensive for frequently accessed e-commerce data.

B+ Tree Indexed Organization

With a B+ Tree, the search requires only a small number of index-level accesses. For 10 million records, a well-designed B+ Tree may require roughly **3–4 index levels**, followed by access to the required data block.

Thus:

$$
\text{Search I/O} \approx 4\text{-}5\text{ block accesses}
$$

instead of hundreds of thousands of block accesses.

Comparison

Feature	**Sequential File**	**B+ Tree Indexed File**
Searching	Slow	Very fast
Search complexity	O(N)	O(log N)
Range queries	Moderate/slow	Very efficient

Feature	Sequential File	B+ Tree Indexed File
Insertion	Easy if appended	Efficient
Deletion	May require reorganization	Efficient
Index storage	None	Additional storage required
Disk I/O	High	Low
Suitability for 10M records	Moderate	**Excellent**

2. Indexing Strategy for Fast Banking Transaction Retrieval

Introduction

A banking system stores a large number of transaction records. Customers and bank employees frequently need to retrieve transactions using an **account number** and a **date range**. Therefore, an efficient indexing strategy is required to reduce search time and disk I/O operations.

Assume the transaction table is:

```
TRANSACTION(
   Transaction_ID,
   Account_No,
   Transaction_Date,
   Transaction_Type,
   Amount,
   Balance
)
```

Recommended Index

The most suitable index is a **composite B+ Tree index** on Account_No and Transaction_Date.

```
CREATE INDEX idx_account_date
ON TRANSACTION(Account_No, Transaction_Date);
```

The index first organizes transactions by Account_No and then by Transaction_Date.

Example Query

Consider the following query:

```
SELECT *
FROM TRANSACTION
WHERE Account_No = 101234
AND Transaction_Date BETWEEN '2026-01-01' AND '2026-01-31';
```

Using the composite index, the database can:

1. Locate the required account quickly.
2. Search only the required date range for that account.
3. Retrieve the matching transaction records.

This avoids scanning the entire transaction table.

Additional Index

A separate B+ Tree index on Transaction_Date can be created if the bank frequently searches transactions across **all accounts** for a particular period.

```
CREATE INDEX idx_transaction_date
ON TRANSACTION(Transaction_Date);
```

For example:

```
SELECT *
FROM TRANSACTION
WHERE Transaction_Date BETWEEN '2026-08-01' AND '2026-08-18';
```

Primary Key Index

Transaction_ID should be the primary key. A unique B+ Tree index can be maintained automatically by the database.

```
ALTER TABLE TRANSACTION
ADD PRIMARY KEY (Transaction_ID);
```

This provides fast retrieval of an individual transaction.

Choice of B+ Tree

A **B+ Tree** is suitable because:

- It provides fast equality searches.
- It efficiently supports date-range queries.

- Leaf nodes are linked, making sequential range retrieval efficient.
- It reduces the number of disk I/O operations.
- It performs well with very large transaction tables.

Cost Analysis

Without indexing, the database may need to perform a **full table scan** to find transactions, resulting in high disk I/O.

With the composite B+ Tree index:

[
Search\ Cost \approx O(\log N) + O(k)
]

where:

- N = total number of transaction records.
- k = number of matching transactions.

Although indexes require additional storage and slightly increase the cost of insertion and deletion, the improvement in transaction retrieval makes the additional cost worthwhile.

3. Multi-Level Indexing for Healthcare Patient Records

Introduction

A healthcare system stores a large number of patient records. These records need to be accessed quickly using **PatientID** as well as **Doctor Name**. Since both fields are frequently used for searching, a multi-level indexing technique can reduce search time and disk I/O.

Assume the patient table is:

```
PATIENT(
    PatientID,
    PatientName,
    Age,
    DoctorName,
    Department,
    Diagnosis,
    AdmissionDate
)
```

Primary Index on PatientID

PatientID is unique for every patient, so a **B+ Tree primary index** should be created.

CREATE UNIQUE INDEX idx_patient_id

ON PATIENT(PatientID);

This index allows the system to quickly locate a patient's complete record using their PatientID.

For example:

SELECT *

FROM PATIENT

WHERE PatientID = 10025;

The search requires only a small number of index-level accesses instead of scanning the entire patient table.

Secondary Index on DoctorName

Since multiple patients may be treated by the same doctor, DoctorName is a **non-unique attribute**. Therefore, a **secondary B+ Tree index** should be created.

CREATE INDEX idx_doctor_name

ON PATIENT(DoctorName);

For example:

SELECT *

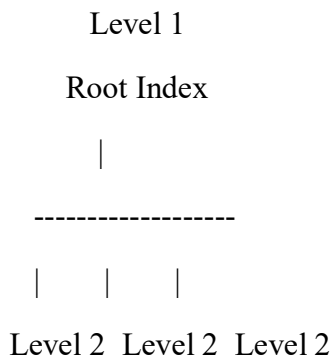
FROM PATIENT

WHERE DoctorName = 'Dr. Kumar';

The index quickly locates all records associated with that doctor.

Multi-Level Index Structure

A multi-level index can be organized as:



Index

| | |

Level 3 Level 3 Level 3

Index

| | |

Data Blocks / Patient Records

The higher levels contain pointers to lower-level index blocks. The lowest-level index points to the actual patient records.

For PatientID, the structure is:

PatientID

↓

B+ Tree Root

↓

Intermediate Index

↓

Leaf Index

↓

Patient Record

A separate secondary B+ Tree can be maintained for DoctorName.

Search Operations

Search by PatientID:

PatientID → Primary B+ Tree → Data Block → Patient Record

Search by DoctorName:

DoctorName → Secondary B+ Tree → Matching Record Pointers → Patient Records

This allows both types of searches to be performed efficiently.

Cost Analysis

Without indexes, searching for a patient may require a **full table scan**, with a cost approximately proportional to the number of records:

[
O(N)
]

With a multi-level B+ Tree index, the search cost becomes approximately:

[
O(\log_B N)
]

where:

- N = number of patient records
- B = number of entries that can fit in an index block

Because each index level contains many pointers, the height of the index remains small even when the number of records is very large.

Advantages

- Fast PatientID-based retrieval.
- Efficient retrieval of all patients treated by a particular doctor.
- Reduces disk I/O.
- Suitable for large healthcare databases.
- Supports both unique and non-unique searches.
- B+ Tree supports efficient range queries if required, such as patients admitted between two dates.

4. B-Tree vs B+ Tree for Dynamic Shipment Records

Introduction

A logistics company continuously adds new shipment records and removes completed or cancelled shipments. Therefore, the file organization must support fast insertion, deletion, and searching without requiring frequent reorganization of the entire database.

Two suitable indexing structures are B-Tree and B+ Tree.

2. B-Tree

A B-Tree stores both keys and record pointers in internal nodes and leaf nodes.

```
        [30 | 60]
       /    |    \
   Records
```

Advantages:

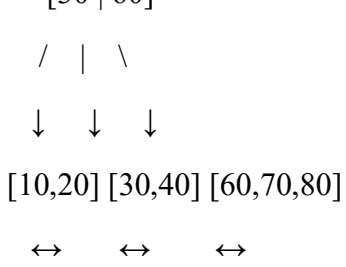
- Supports fast search, insertion, and deletion.
- Records can be accessed from internal or leaf nodes.
- Keeps the tree balanced after updates.

Disadvantages:

- Internal nodes store record pointers, reducing the number of keys per node.
- Range queries are less efficient because leaf nodes are not necessarily linked.
- More disk I/O may be required for sequential retrieval.

B+ Tree

A B+ Tree stores search keys in internal nodes, while actual record pointers are stored at the **leaf level**. The leaf nodes are linked.



Advantages:

- High fan-out reduces tree height.
- Efficient search, insertion, and deletion.
- Linked leaf nodes make range queries very efficient.
- Better disk-block utilization.
- Suitable for large databases with frequent updates.

Comparison

Feature	**B-Tree**	**B+ Tree**
Search	Fast	**Very fast**
Insertion	Efficient	**Efficient**
Deletion	Efficient	**Efficient**
Range queries	Less efficient	**Highly efficient**

Feature	B-Tree	B+ Tree
Leaf nodes linked	Usually no	**Yes**
Fan-out	Lower	**Higher**
Disk I/O	Higher	**Lower**
Database suitability	Good	**Excellent**

Cost Analysis

For N shipment records, both structures provide approximately:

[
O(\log N)
]

for search, insertion, and deletion.

However, B+ Trees generally have a **smaller height** because internal nodes contain only keys and child pointers. Therefore, fewer disk accesses are required.

For example, when a shipment is inserted:

New Shipment

↓

Search B+ Tree

↓

Locate Leaf

↓

Insert Record

↓

Split Leaf if Necessary

When a shipment is deleted, the corresponding leaf is updated and the tree remains balanced through redistribution or merging when required.

Recommendation

The **B+ Tree is more suitable** for the logistics company.

Shipment systems commonly require operations such as:

- Find a shipment using ShipmentID.

- Find shipments for a particular customer.
- Retrieve shipments within a date range.
- Find shipments between two locations.
- Insert new shipment records.
- Delete completed or cancelled shipments.

B+ Tree supports these operations efficiently, especially when range-based queries are required.

5. Hashing Scheme for an HR System with 5000 Employees

Introduction

An HR system needs to store and retrieve records of **5000 employees**. Employee records are frequently accessed using a unique **Employee ID**. A hashing technique can provide fast direct access to employee records.

Assume the employee table contains:

EMPLOYEE(

Employee_ID,

Employee_Name,

Department,

Salary,

Designation

)

Proposed Hashing Scheme

Use Employee_ID as the search key.

A simple hash function can be:

[
h(Employee_ID) = Employee_ID \mod 100
]

This creates **100 primary buckets**.

For example:

Employee ID = 12547

h(12547) = 12547 mod 100 = 47

Therefore, the employee record is stored in **Bucket 47**.

Static Hashing

In static hashing, the number of buckets is fixed when the file is created.

For 5000 employees, we can initially create **100 buckets**, with approximately:

[
\frac{5000}{100}=50
]

employees per bucket.

Advantages

- Simple to implement.
- Fast direct access.
- Low implementation overhead.
- Suitable when the number of employees remains relatively stable.

Disadvantages

- The number of buckets cannot easily grow.
- New employees may cause overflow.
- Overflow chains increase search time.
- Periodic reorganization may be required if the employee database grows significantly.

Extendible Hashing

Extendible hashing is a **dynamic hashing technique** in which the number of buckets can increase or decrease according to the amount of data.

Initially, the directory may contain a small number of entries:

Directory

|

+---- Bucket 00

+---- Bucket 01

+---- Bucket 10

+---- Bucket 11

When a bucket becomes full, it is split and the directory can be expanded.

Before:

Bucket 01 → Full

After:

Bucket 001

Bucket 101

This reduces long overflow chains and maintains efficient access as employees are added.

Static vs. Extendible Hashing

Feature	**Static Hashing**	**Extendible Hashing**
Number of buckets	Fixed	Dynamic
Implementation	Simple	More complex
Search	Fast	Fast
Insertion	May cause overflow	Bucket splitting
Growth handling	Poor	**Excellent**
Overflow chains	Possible	Minimized
Storage overhead	Low	Slightly higher
Suitable for changing employee count	Moderate	**Excellent**

Cost Analysis

For static hashing, if 100 buckets are used:

$$
Average\ records/bucket = \frac{5000}{100}=50
$$

If the employee count increases significantly, buckets may become overloaded and overflow blocks may be required. Extendible hashing dynamically splits buckets when they become full. Therefore, the average search remains close to **O(1)** while avoiding excessive overflow chains. Although extendible hashing requires additional directory space, this overhead is small compared with the performance benefit for a growing HR database.

Recommendation

For an HR system with exactly 5000 employees and little expected growth, static hashing is sufficient because it is simple and economical. However, an HR database normally grows as employees are recruited. Therefore, extendible hashing is the better long-term choice because it dynamically handles insertions and prevents excessive overflow.

6. **Combined Index Structure for Airline Reservation System**

Introduction

An airline reservation system stores a large number of flight and reservation records. It must support:

1. **Point queries** – finding a specific flight using its Flight_No.
2. **Range queries** – finding flights operating within a particular date range.

A combined indexing strategy using **B+ Tree indexes** is suitable for these requirements.

Table Structure

Assume the flight table is:

```
FLIGHT(
   Flight_No,
   Flight_Date,
   Source,
   Destination,
   Departure_Time,
   Arrival_Time,
   Available_Seats
)
```

Index for Point Queries

Create a **unique B+ Tree index** on Flight_No.

```
CREATE UNIQUE INDEX idx_flight_no
```

ON FLIGHT(Flight_No);

For a query such as:

SELECT *

FROM FLIGHT

WHERE Flight_No = 'AI202';

The B+ Tree directly locates the required flight.

The search cost is approximately:

[
O(\log N)
]

where N is the number of flight records.

Index for Date Range Queries

Create another **B+ Tree index** on Flight_Date.

CREATE INDEX idx_flight_date

ON FLIGHT(Flight_Date);

This supports queries such as:

SELECT *

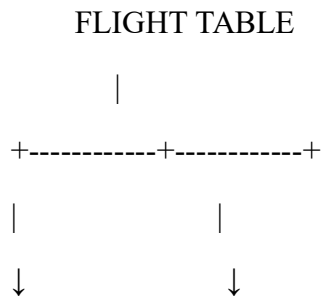
FROM FLIGHT

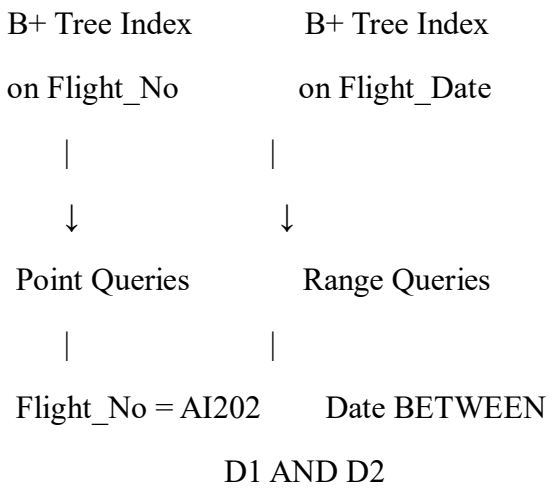
WHERE Flight_Date BETWEEN '2026-08-01' AND '2026-08-31';

The B+ Tree locates the first matching date and then uses its linked leaf nodes to efficiently retrieve the remaining dates.

Combined Index Structure

The proposed structure is:





Why B+ Tree?

B+ Tree is appropriate for both requirements because:

- It provides fast point searches.
- It supports efficient range searches.
- Leaf nodes are linked for sequential traversal.
- It has high fan-out, reducing tree height.
- It minimizes disk I/O.
- It performs well for large and frequently accessed databases.

Cost Analysis

Operation	Index Used	Approximate Cost
Find one flight by number	B+ Tree on Flight_No	O(log N)
Find flights within date range	B+ Tree on Flight_Date	O(log N + K)
Insert flight	Both indexes updated	O(log N)
Delete flight	Both indexes updated	O(log N)

Here, K represents the number of records returned by a range query.

Although maintaining two indexes requires additional storage and causes a small overhead during insertion and deletion, it greatly improves query performance.

Alternative: Composite Index

If queries usually contain **both Flight_No and Flight_Date**, a composite B+ Tree index can also be created:

CREATE INDEX idx_flight_date_no

ON FLIGHT(Flight_Date, Flight_No);

However, for the given requirement, **two separate indexes are preferable** because point queries and date-range queries are independent.

7. File Organization and Secondary Indexing for University Student Database

Introduction

A university database stores a large number of student records. The system should support fast queries based on **Department** and **CGPA range**. An appropriate file organization combined with secondary indexes can significantly reduce search time and disk I/O.

Assume the student table is:

STUDENT(

Student_ID,

Student_Name,

Department,

CGPA,

Year,

Email

)

Recommended File Organization

Use a **heap file organization** for storing the actual student records.

In heap organization, records are stored in no particular order. New student records can be inserted into available space without reorganizing the entire file.

Student File

| Record 1 | Record 2 | Record 3 | ... | Record N |

Heap organization is suitable because student records may be frequently inserted, updated, or deleted.

Secondary Index on Department

Department is not unique because many students belong to the same department. Therefore, create a **secondary B+ Tree index** on Department.

```
CREATE INDEX idx_department

ON STUDENT(Department);
```

For example:

```
SELECT *

FROM STUDENT

WHERE Department = 'CSE';
```

The index quickly identifies the records belonging to the CSE department without scanning the complete student file.

Secondary Index on CGPA

Create another **secondary B+ Tree index** on CGPA.

```
CREATE INDEX idx_cgpa

ON STUDENT(CGPA);
```

This is especially useful for range queries such as:

```
SELECT *

FROM STUDENT

WHERE CGPA BETWEEN 8.0 AND 9.0;
```

The B+ Tree locates the first CGPA value in the range and then follows the linked leaf nodes to retrieve the remaining matching records.

Combined Query

The system may also need queries involving both department and CGPA:

```
SELECT *

FROM STUDENT
```

WHERE Department = 'CSE'

AND CGPA BETWEEN 8.0 AND 9.0;

The database can use the Department index to locate CSE students and then apply the CGPA condition.

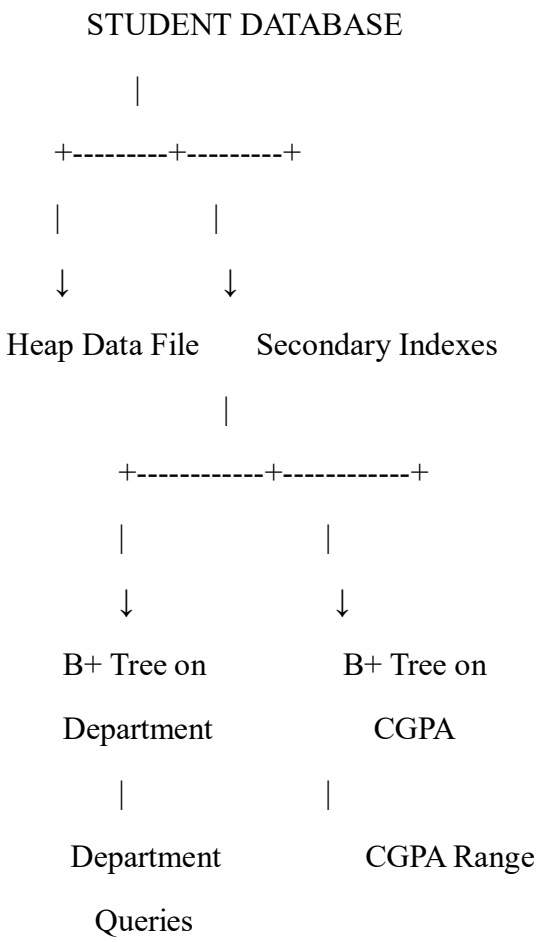
If this combined query is very frequent, a **composite B+ Tree index** can be created:

CREATE INDEX idx_dept_cgpa

ON STUDENT(Department, CGPA);

This index is particularly efficient because it first groups records by department and then orders the CGPA values within each department.

Recommended Structure

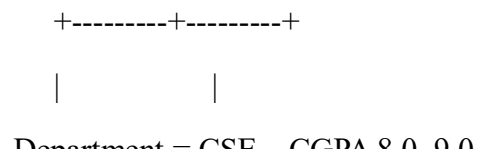


For frequently combined queries:

Composite B+ Tree

(Department, CGPA)

|



Cost Analysis

Assume there are N student records.

Without indexing, a department or CGPA query may require a full table scan:

$$O(N)$$

With a B+ Tree secondary index, searching is approximately:

$$O(\log N)$$

For a range query returning K records:

$$O(\log N + K)$$

The indexes require additional disk space and slightly increase the cost of insertion, deletion, and updates. However, the reduction in query time makes this additional cost worthwhile.

8. Disk Block Utilization for Heap, Sorted, and Hashed File Organizations

Given Information

- Number of records = **100,000**
- Record size = **50 bytes**
- Block size = **4 KB = 4096 bytes**

Records per Block

The number of complete records that can fit into one block is:

$$\text{Records per block}= \left\lfloor\frac{4096}{50}\right\rfloor =81$$

Therefore, **81 records** can fit in each block.

Space used by 81 records:

$$
81 \times 50 = 4050\text{ bytes}
$$

Unused space per completely filled block:

$$
4096-4050=46\text{ bytes}
$$

Block utilization:

$$
\frac{4050}{4096}\times100
\approx98.88\%
$$

Number of Blocks Required

$$
\text{Blocks}=
\left\lceil\frac{100000}{81}\right\rceil
=1235\text{ blocks}
$$

The first **1234 blocks** contain 81 records each.

Records in the final block:

$$
100000-(1234\times81)=46
$$

Space used in the final block:

$$
46\times50=2300\text{ bytes}
$$

Final-block utilization:

$$
\frac{2300}{4096}\times100
\approx56.15\%
$$

Overall storage used for records:

$$
100000\times50=5,000,000\text{ bytes}
$$

Total allocated block space:

$$
1235\times4096=5,058,560\text{ bytes}
$$

Overall utilization:

$$
\frac{5,000,000}{5,058,560}\times100
\approx98.84%
$$

Comparison of File Organizations

File Organization	Blocks Required	Utilization	Main Advantage
Heap	1235	≈ 98.84%	Fast insertion
Sorted	1235	≈ 98.84%	Efficient sequential/range access
Hashed	1235*	≈ 98.84%*	Fast equality search

*Assuming an ideal hash distribution with no significant overflow blocks.

Heap File Organization

In a **Heap file**, records are stored in no particular order.

- Requires approximately **1235 blocks**.
- Block utilization is approximately **98.84%**.
- New records can be inserted easily.
- Searching may require scanning many blocks.

Best suited for: frequent insertions and general-purpose storage.

Sorted File Organization

In a **Sorted file**, records are maintained according to a search key.

- Requires approximately **1235 blocks**.
- Utilization is approximately **98.84%**.
- Provides efficient sequential and range-based access.
- Insertion can be expensive because records may need to be repositioned.

Best suited for: range queries and ordered data retrieval.

Hashed File Organization

In a **Hashed file**, a hash function determines the block in which a record is stored.

Ideally, with uniform distribution:

- Requires approximately **1235 blocks**.
- Utilization is approximately **98.84%**.
- Provides very fast equality searches.
- Poor hash distribution can create overflow blocks and reduce utilization.

Best suited for: exact-match queries such as searching by Student_ID or Employee_ID.

9. Composite Indexing Strategy for 500 Million Posts

Introduction

A social media platform stores **500 million posts**. Users need to retrieve posts based on:

- user_id – to display a user's posts
- timestamp – to retrieve recent or historical posts
- hashtag – to search posts containing a particular hashtag

Because the data volume is extremely large, a carefully designed **composite indexing strategy** is required.

Assume the table is:

```
POST(
   post_id,
   user_id,
   timestamp,
   hashtag,
   content
)
```

Primary Index on Post ID

post_id should be the primary key.

```
CREATE UNIQUE INDEX idx_post_id
ON POST(post_id);
```

This provides fast retrieval of an individual post.

Composite Index on User and Timestamp

The most important composite index should be:

```
CREATE INDEX idx_user_time
ON POST(user_id, timestamp);
```

This index first groups posts by user_id and then sorts them by timestamp.

It efficiently supports queries such as:

```
SELECT *
FROM POST
WHERE user_id = 101
AND timestamp BETWEEN '2026-08-01' AND '2026-08-18';
```

It is also useful for displaying a user's latest posts:

```
SELECT *
FROM POST
WHERE user_id = 101
ORDER BY timestamp DESC;
```

Index for Hashtag Searches

A separate index should be created for hashtags:

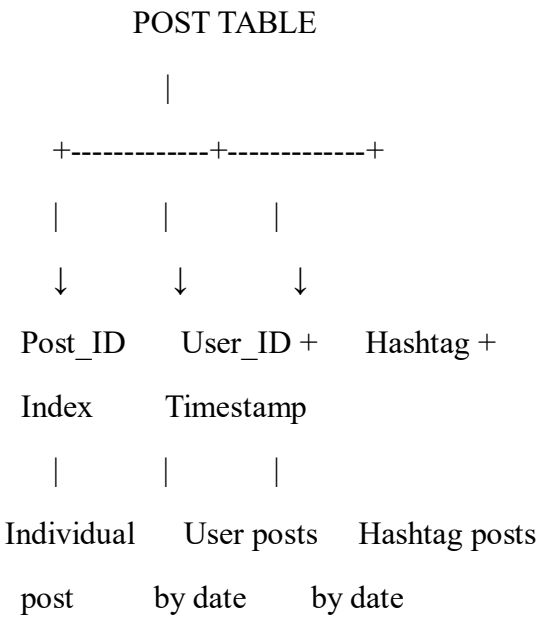
```
CREATE INDEX idx_hashtag_time
ON POST(hashtag, timestamp);
```

This supports queries such as:

```
SELECT *
FROM POST
WHERE hashtag = '#technology'
AND timestamp BETWEEN '2026-08-01' AND '2026-08-18';
```

The timestamp as the second column makes recent-post and date-range searches within a hashtag efficient.

Recommended Index Structure



Why Composite Indexes?

A composite index combines multiple search attributes into a single index.

For:

(user_id, timestamp)

the database can first locate the user_id and then efficiently scan the required timestamp range.

Similarly:

(hashtag, timestamp)

allows the database to locate a hashtag and then retrieve posts in timestamp order.

Index Ordering

The order of columns is important.

Recommended:

(user_id, timestamp)

(hashtag, timestamp)

The frequently filtered attribute comes first, followed by the range/sorting attribute.

For example, (user_id, timestamp) is better than (timestamp, user_id) for queries that specify a particular user and then request posts within a date range.

Handling 500 Million Records

Maintaining indexes over 500 million records can make the indexes very large. Therefore, the system should additionally use:

- **B+ Tree indexes** for ordered and range-based access.
- **Partitioning by time** such as month or year to reduce the amount of data searched.
- **Sharding by user_id or another suitable distribution key** for horizontal scalability.
- Separate hashtag indexing because hashtags are many-to-many with users.
- Index only columns required for frequent queries to control storage and update cost.

Cost Analysis

For a B+ Tree index, point lookup is approximately:

[
O(\log N)
]

For a range query returning K records:

[
O(\log N + K)
]

Without indexes, a query may require scanning a large portion of the **500 million records**, resulting in very high disk I/O.

The composite indexes require additional storage and increase the cost of inserting new posts because each new post must update the relevant indexes. However, the reduction in query time is essential for a social media platform.

10. Performance Comparison of Heap File, Sorted File, and Hashed File

Introduction

A supermarket billing system may generate around 1 million transactions per day. Therefore, an efficient file organization is required to perform insert, delete, and search operations quickly. The three common file organizations are Heap File, Sorted File, and Hashed File.

Performance Comparison

Operation	Heap File	Sorted File	Hashed File
Insertion	Very Fast	Slow	Very Fast
Deletion	Slow	Moderate/Slow	Very Fast
Search	Slow	Fast	Very Fast
Range Search	Slow	Very Fast	Slow
Data Arrangement	Unordered	Sorted	Hash-based
Best Application	Frequent insertion	Range queries	Exact-value search

Heap File

In a heap file, transactions are stored in no particular order.

- **Insertion:** Very fast because a new transaction can be added directly to available space.
- **Deletion:** Slow because the transaction must first be located.
- **Search:** Slow because a sequential search may be required.
- **Advantage:** Suitable for systems with frequent transaction insertion.
- **Disadvantage:** Searching a large number of transactions is inefficient.

Sorted File

In a sorted file, records are maintained according to a particular field, such as **Transaction ID** or **Transaction Date**.

- **Insertion:** Slow because the correct position must be maintained.
- **Deletion:** Relatively slow because the sorted order must be preserved.
- **Search:** Fast because binary search can be used.
- **Advantage:** Excellent for range queries such as finding transactions between two dates.
- **Disadvantage:** Frequent insertions and deletions are expensive.

Hashed File

In a hashed file, a **hash function** determines the location of each transaction.

- **Insertion:** Very fast on average.
- **Deletion:** Very fast when the transaction ID is known.
- **Search:** Very fast for exact-value searches.
- **Advantage:** Ideal for quickly finding a particular bill using its transaction ID.
- **Disadvantage:** Not suitable for range queries.

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation